RecuVeicol

Gestione del soccorso stradale

Project work · ITS Developer 4.0
Novembre 2026



Team
Vincenzo Minervini · Francesco Valeriano
Giuseppe Cangellosi · Marika Sancilio · Marco Colucci

APERTURA

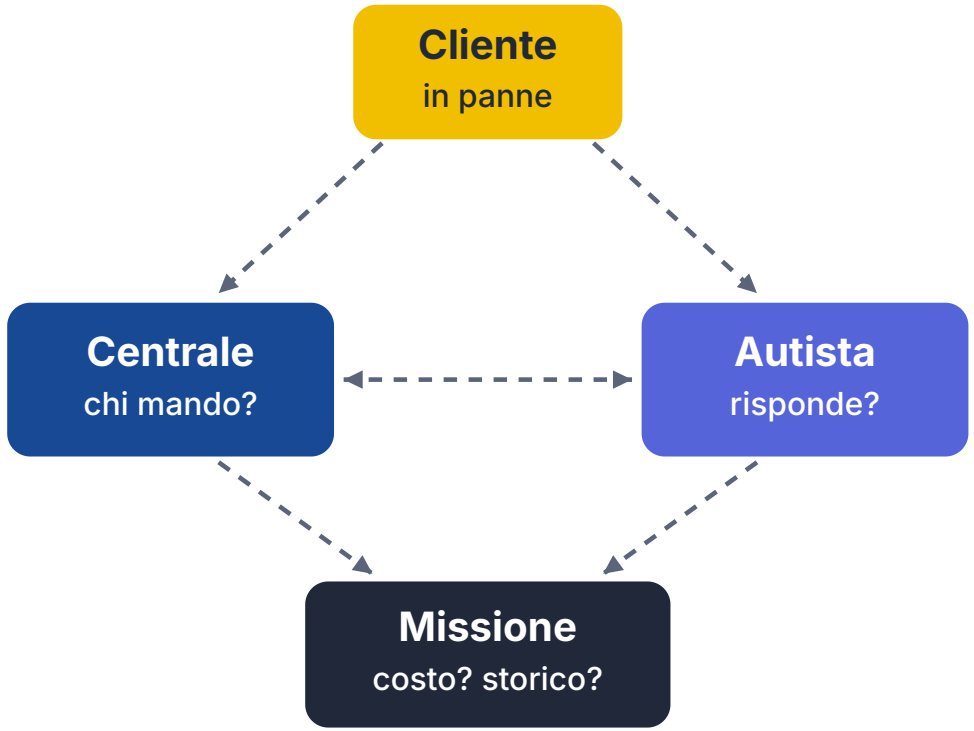
Team e ruoli

Membro	Area	Responsabili
Membro 1	Backend	Vincenzo Minervini
Membro 2	Frontend	Francesco Valeriano, Giuseppe Cangellosi, Marika Sancilio
Membro 3	Chatbot	Marco Colucci
Membro 4	Mappe	Marco Colucci
Membro 5	Notifiche	Marika Sancilio, Francesco Valeriano, Giuseppe Cangellosi

Cinque aree, un contratto API condiviso: ogni componente parla agli altri solo tramite le API.

Il problema

- Chi resta in panne vuole chiedere aiuto subito, anche dal telefono
- La centrale deve trovare un autista libero e vicino
- Se l'autista non risponde, la richiesta non deve fermarsi
- Il cliente vuole sapere chi sta arrivando
- Costi e storico devono restare verificabili



La soluzione

- **Chat Telegram**: il cliente apre la richiesta con la targa
- **Web centrale**: anagrafiche, missioni, proposta all'autista
- **Web autista**: accetta, vede la mappa, chiude con i km
- **Backend unico**: regole, costi e permessi sul server
- **Notifiche**: il cliente è avvisato in chat



Sito pubblico RecuVeicol: da qui si apre il bot

APERTURA

Attori e ruoli

Cliente

- Usa Telegram, senza login
- Scrive targa, luogo e problema
- Chiede lo stato, riceve gli avvisi

Centrale

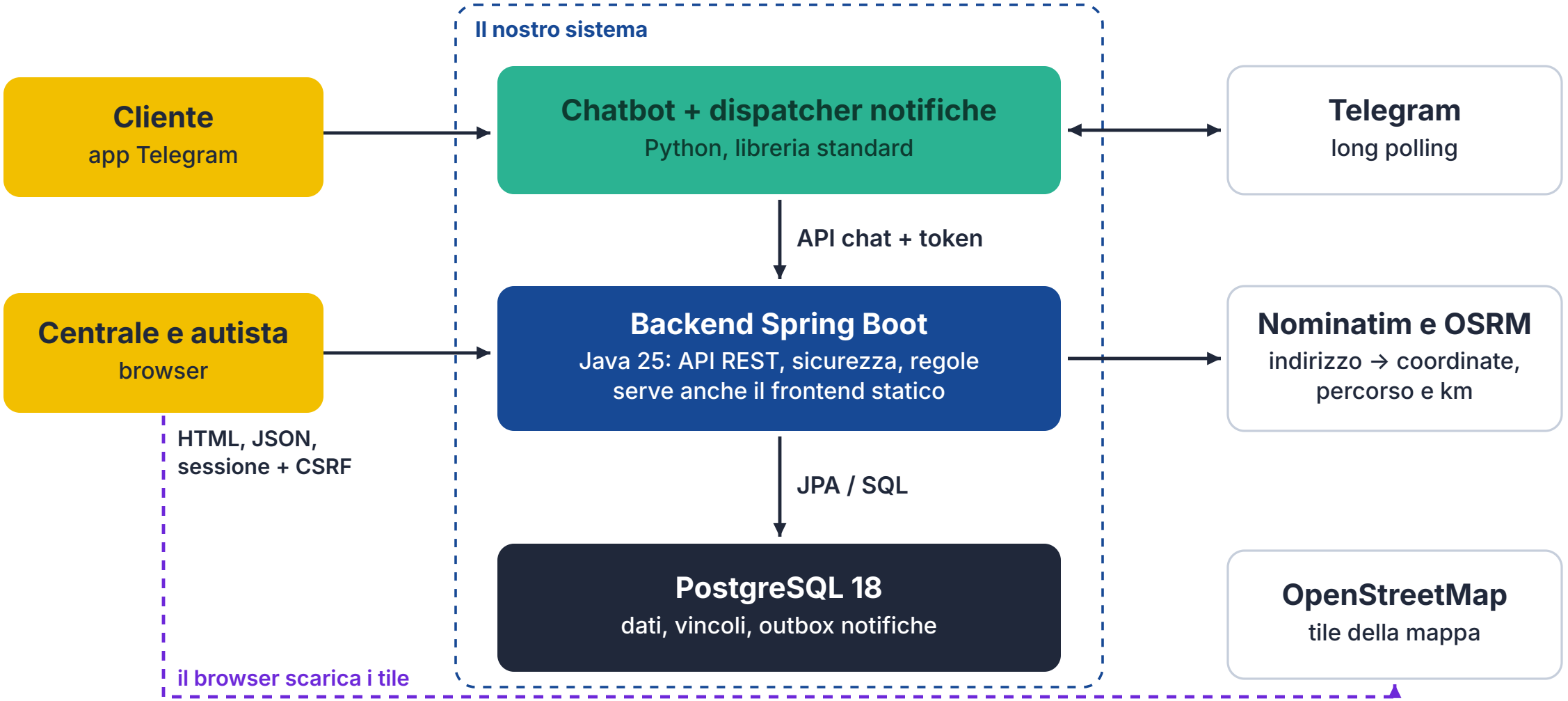
- Ruolo **RESPONSABILE**
- Gestisce anagrafiche e missioni
- Propone, corregge, rettifica il costo

Autista

- Ruolo **AUTISTA**
- Accetta o rifiuta entro 10 minuti
- Segnala, chiude e vede i guadagni

I contatti del cliente sono visibili all'autista solo dopo l'accettazione.

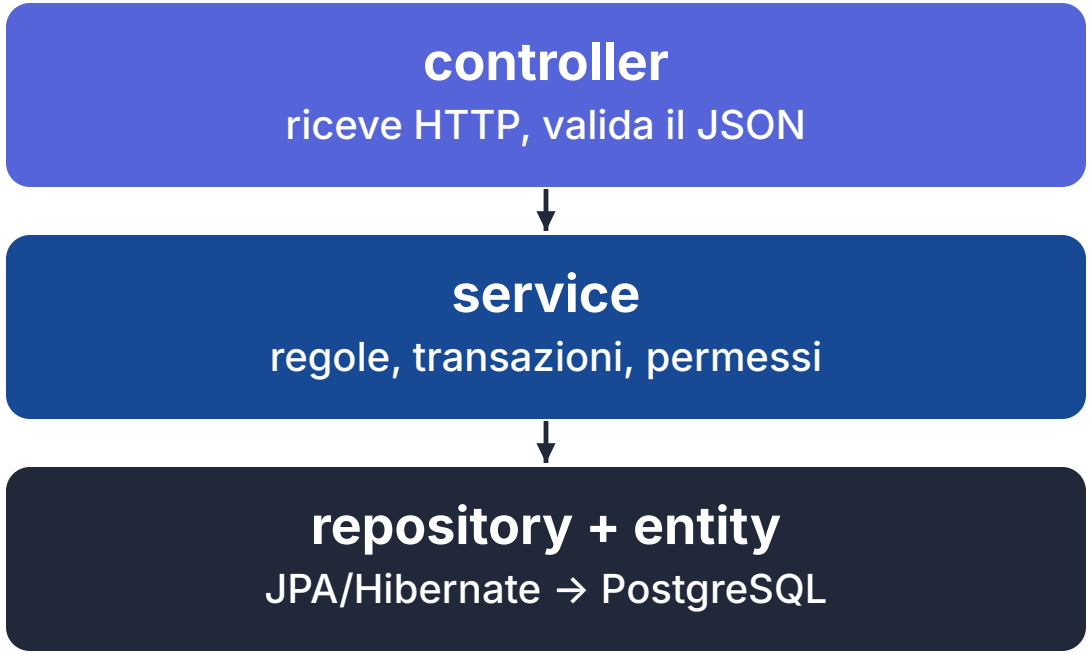
Architettura generale



Tecnologie: cosa e perché

Livello	Tecnologia	Linguaggio	Perché
Frontend	HTML, CSS, Bootstrap 5.3, Leaflet 1.9	JavaScript	Nessuna build: lo serve Spring Boot
Backend	Spring Boot 4.1.2, Security, Data JPA	Java 25	Sicurezza, validazione e dati già integrati
Database	PostgreSQL 18, script numerati	SQL	Vincoli e transazioni proteggono le missioni
Chatbot	Telegram Bot API, long polling	Python 3.10+	Solo libreria standard, nessun indirizzo pubblico
Mappe	Nominatim, OSRM, OpenStreetMap	Java, JS	Open source, senza chiavi a pagamento
Notifiche	Outbox su PostgreSQL + dispatcher	Java, Python	Nessun messaggio perso in silenzio
Build	Maven Wrapper 3.9.9, GitHub Actions, OpenAPI 3.0.3	XML, YAML	Stessa build per tutti, in locale e in CI

Backend: Java, Spring Boot e Maven



I DTO evitano di esporre le entità. Maven Wrapper: stessa versione per tutti.

Perché Java e Spring Boot

- Sicurezza integrata: sessione, CSRF, ruoli
- Validazione dichiarativa dei dati in ingresso
- JPA/Hibernate: lock e transazioni pronti

Perché Maven

- Dipendenze in un unico **pom.xml**
- Il wrapper non richiede installazioni

Database: PostgreSQL e script numerati

Perché un database relazionale

- Chiavi esterne: niente missioni senza cliente
- Vincoli CHECK: stati e importi sempre validi
- Transazioni: missione e notifica insieme

Perché PostgreSQL

- **Indice parziale**: un autista, una sola missione attiva
- **SKIP LOCKED** per la coda delle notifiche

Script in ordine

01	schema
02	dati demo (seed)
05-06	richieste da chat
07	sicurezza
08	stato di invio notifiche
09	vincoli e indici
10	fascia notturna 22:00

Hibernate solo in **validate**: se manca uno script, l'app non parte.

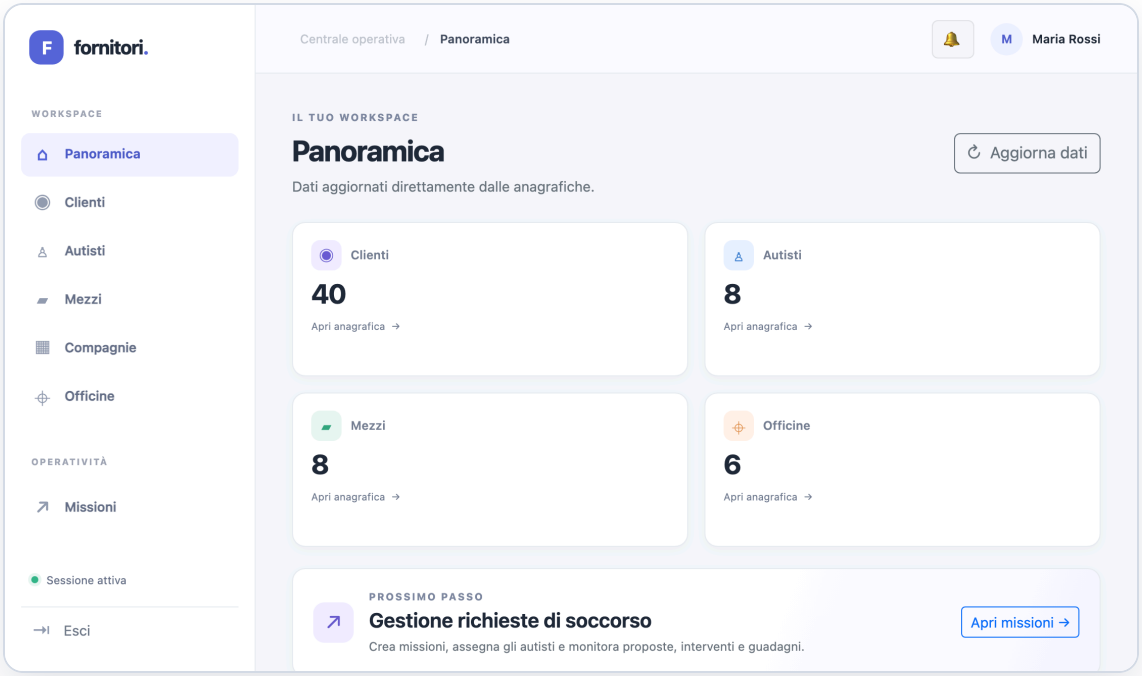
Frontend: JavaScript senza framework e Leaflet

Perché HTML, CSS e JavaScript

- Nessuna compilazione né Node.js
- Servito da Spring Boot: stessa origine, sessione semplice
- Bootstrap per un layout pronto e responsive

Perché Leaflet

- Libreria open source per mappe
- Disegna il percorso GeoJSON di OSRM
- Nessuna chiave a pagamento



Panoramica della centrale, schermata reale

Chatbot e notifiche: Python, Telegram, outbox

Perché Python

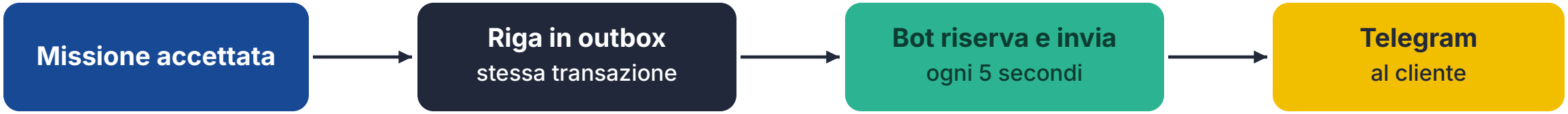
- Solo libreria standard
- Avvio immediato, senza installazioni
- Dialogo a stati facile da leggere

Telegram, non WhatsApp

- API gratuita e aperta
- Nessuna approvazione Meta
- Adatta alla demo: niente server pubblico

Perché l'outbox

- Evento salvato con la missione
- Nessun messaggio perso in silenzio
- Nessun doppio invio tra dispatcher



Caso limite noto: se il bot si ferma tra invio ed esito, quel messaggio può ripartire una volta.

Integrazione: componenti e contratti

Componenti separati

- Ognuno si può sviluppare e provare da solo
- Il bot ha anche un backend finto per la console
- Regole in un solo posto: il server

Nominatim e OSRM

- Open source e gratuiti
- Nessuna chiave a pagamento
- Chiamati dal backend, con timeout

```
POST /api/v1/chat/lookup-targa
{ "sessionId": "…", "targa": "PW660SW" }

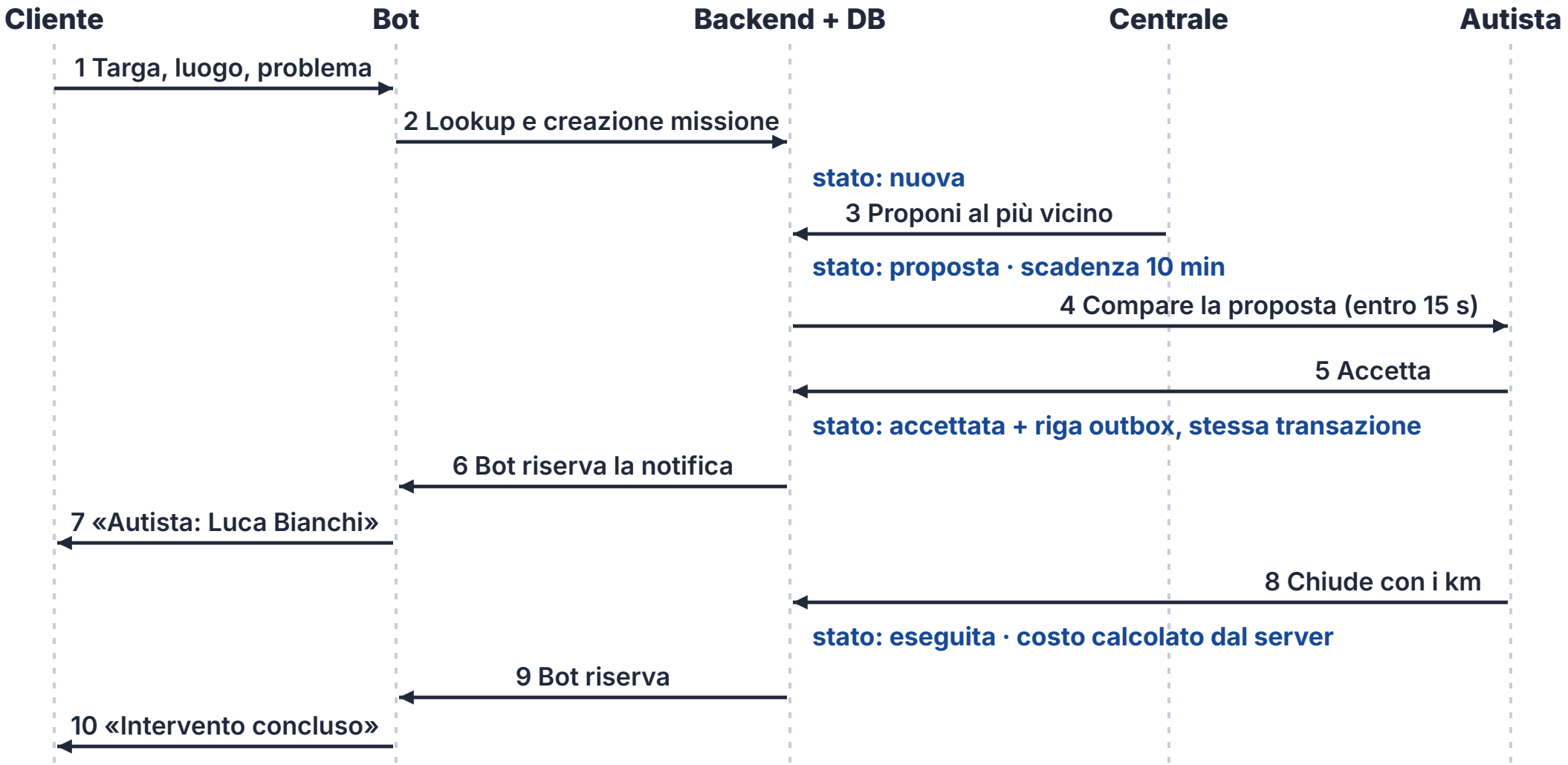
200 OK
{ "verificato": true, "clienteId": 1,
  "clienteNome": "Lucia Gallo",
  "compagniaNome": "Polizza Sicura…" }
```

34
operazioni attive

4
progettate, fuori ambito

OpenAPI
unica fonte

Flusso end-to-end di una missione



Membro 1

Backend

Vincenzo Minervini

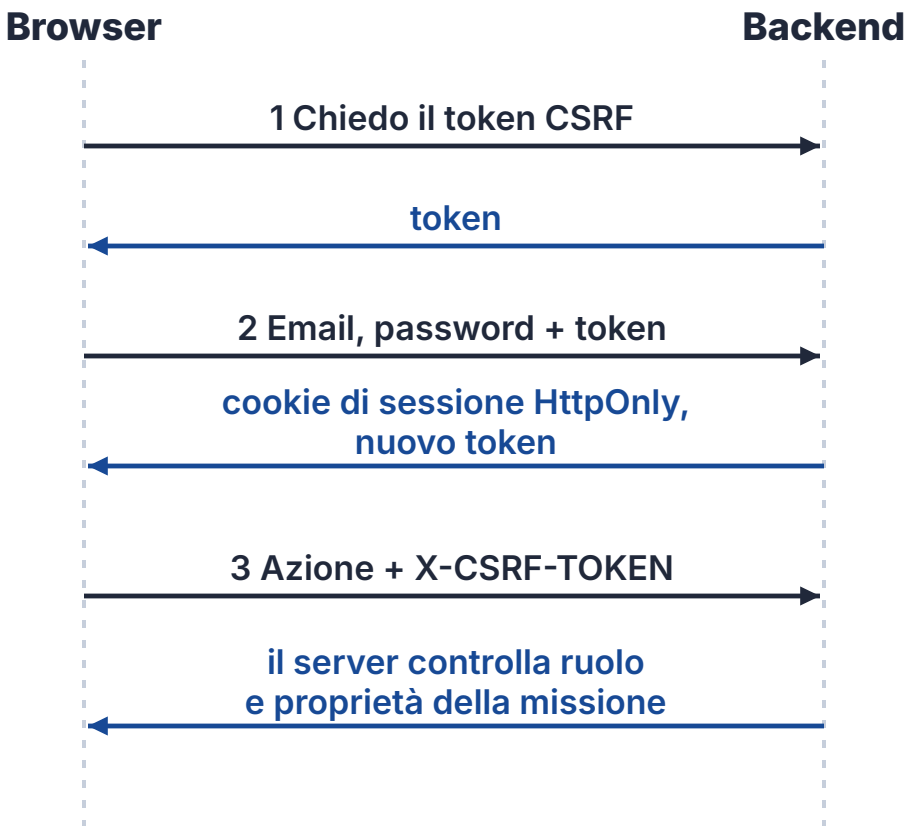
API · sicurezza · logica delle missioni · tariffa notturna · scadenze · assegnazione al più vicino · database

API: chi può chiamare cosa

Area	Op.	A cosa serve	Chi accede
Autenticazione	4	Login, profilo, logout, CSRF	Pubblico, poi sessione
Anagrafiche	6	Clienti, autisti, mezzi, compagnie, officine	Centrale
Missioni e dashboard	16	Ciclo missione, note, costi, storico	Centrale o autista assegnato
Assegnazione	2	Proposta al più vicino, avvisi	Centrale
Chat e notifiche	5	Targa, missione, stato, invio	Bot, con token di servizio
Mappe	1	Percorso e km stimati	Centrale o autista assegnato

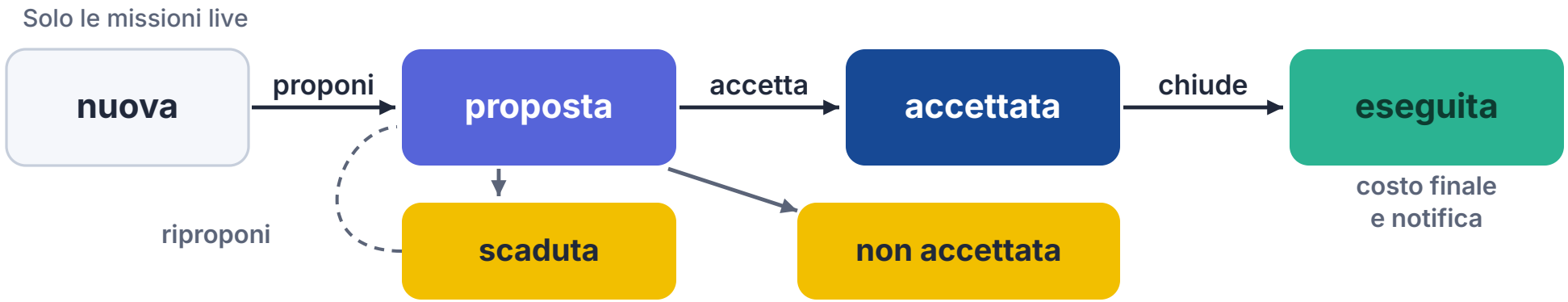
Errori sempre uguali: `{ status, error, message }`

Sicurezza: login, CSRF e ruoli



- **Password**: solo hash BCrypt
- **CSRF**: una pagina esterna non può agire al posto tuo
- **Ruoli**: RESPONSABILE e AUTISTA, verificati dal server
- **Limiti**: 20 login al minuto per IP, 10 falliti per account
- **Bot**: token di servizio separato

Logica delle missioni



Scadenze

- Proposta valida 10 minuti
- Scheduler ogni 30 secondi
- Controllo anche alla risposta

Tariffa

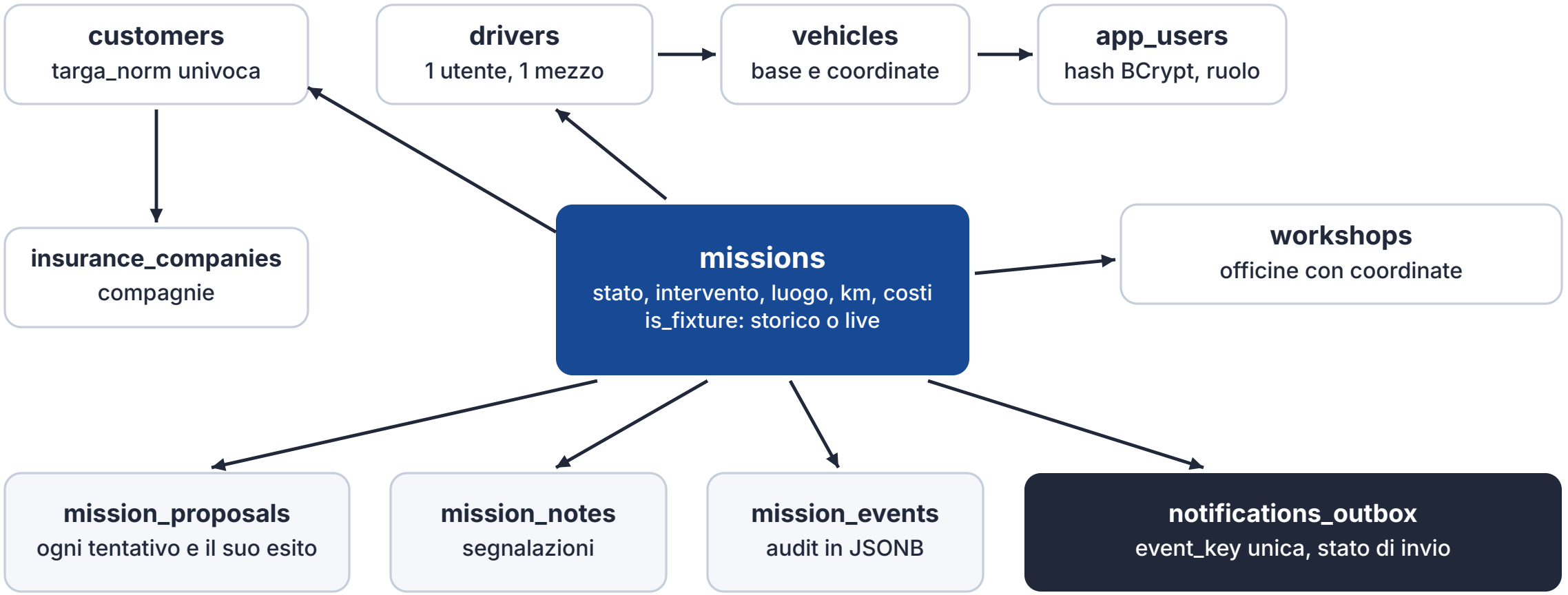
- €50 + €1,20 al km
- +25% dalle 22:00 alle 08:00
- Conta l'ora di accettazione

Al più vicino

- Autisti liberi, in linea d'aria
- Esclude chi ha già risposto
- Se è occupato, passa al prossimo

Fascia 22:00–08:00: scelta del team. Il PDF del docente indica 20:00–8:00, da confermare col docente.

Database: schema semplificato



14 tabelle in totale: qui mancano pricing_config, chat_mission_requests, chat_lookup_grants

Perché l'abbiamo fatto così

Organizzazione dei file

- **controller**, **service**, **repository**
- **dto** separati dalle **entity**
- Script SQL numerati

Pattern

- Livelli con compiti distinti
- Lock pessimistico sulle missioni
- Clock iniettato, testabile nel tempo

Formato dei dati

- JSON in API, errori uniformi
- NUMERIC e BigDecimal per il denaro
- TIMESTAMPTZ, JSONB per l'audit

Membro 2

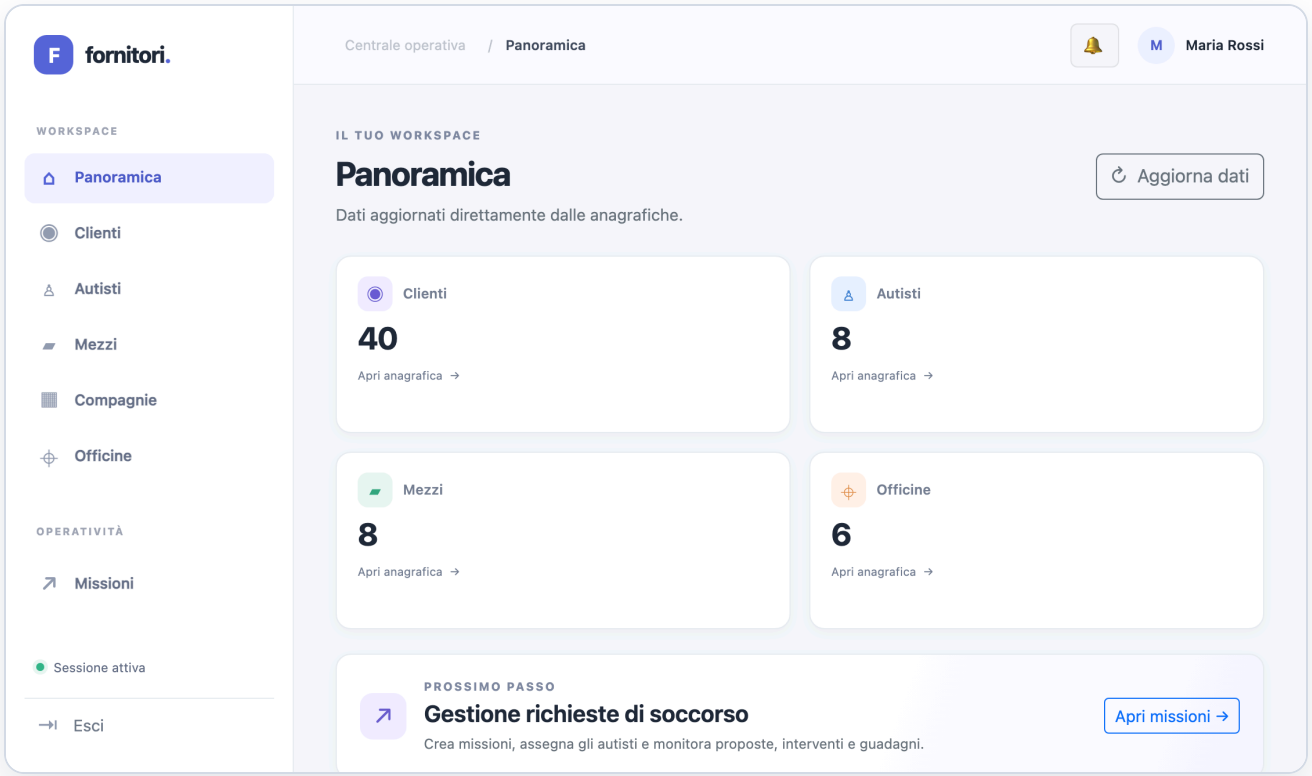
Frontend

Francesco Valeriano · Giuseppe Cangellosi · Marika Sancilio

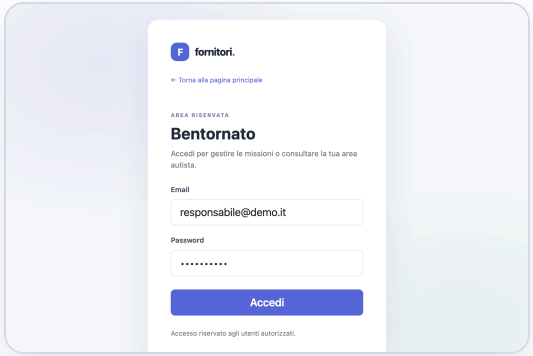
Pagina pubblica · login · centrale operativa · ciclo della missione · area autista

FRONTEND

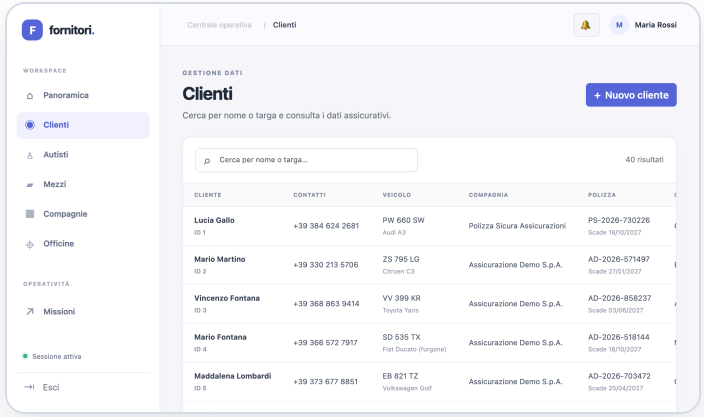
Centrale: accesso e panoramica



Panoramica: conteggi reali dalle anagrafiche



Login

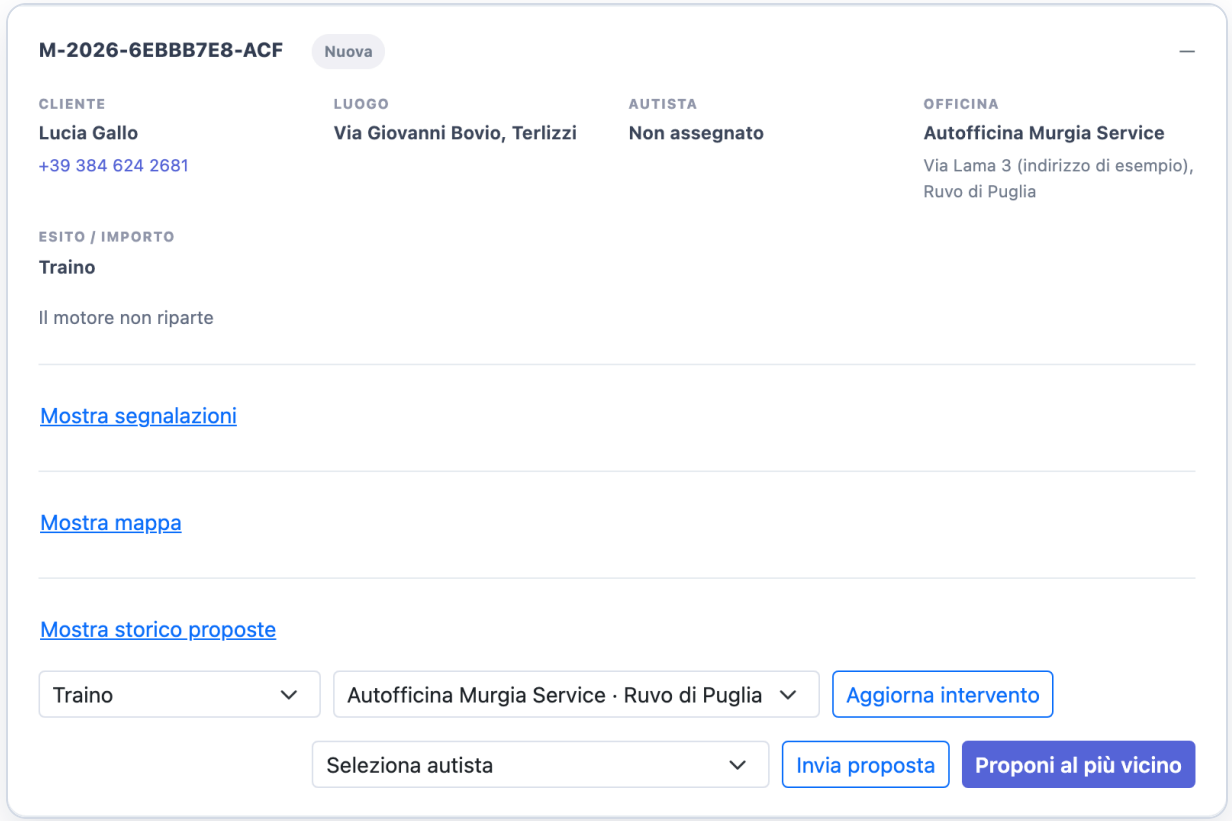


Anagrafiche con ricerca

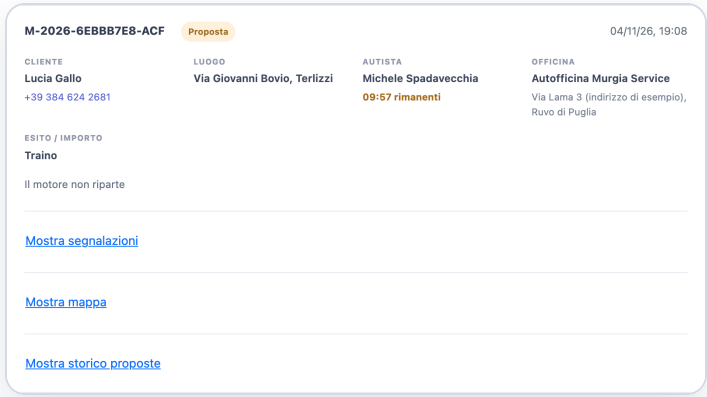
Dati letti dalle API, mai simulati. Errori del server in chiaro, come «Targa già presente».

FRONTEND

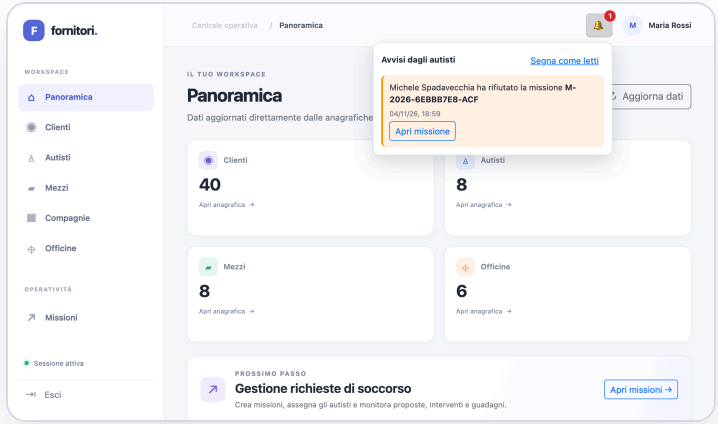
Centrale: il ciclo della missione



1 · Missione nuova dal bot: si propone al più vicino



2 · Proposta: conto alla rovescia di 10 minuti



3 · Rifiuto: la campanella avvisa la centrale

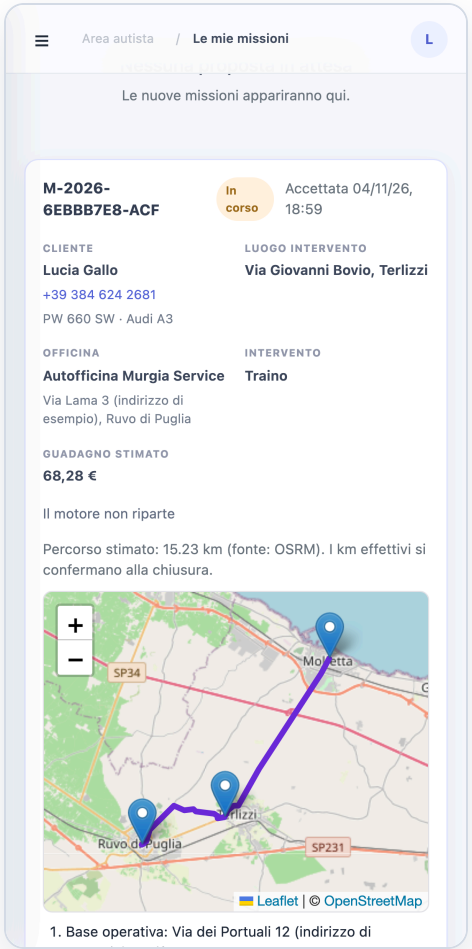
Riproposta con storico. Intervento e officina correggibili. Costo rettificabile con motivo.

FRONTEND

Area autista: dal telefono

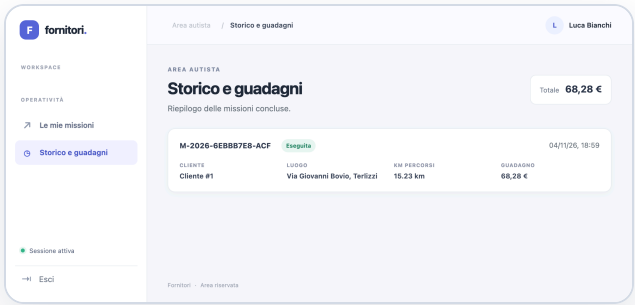


Proposta: contatti nascosti



Dopo aver accettato

- Proposta con tempo residuo
- Guadagno stimato e mappa
- **Accetta** o **Rifiuta**
- Segnalazioni alla centrale
- Chiusura con km proposti



Storico e guadagni

Perché l'abbiamo fatto così

Organizzazione dei file

- Un **index.html**, un **app.js**
- **styles.css** per l'area riservata
- **public-site.css** per il sito

Pattern

- Viste mostrate e nascoste via JS
- Stato in memoria, aggiornato dalle API
- Polling ogni 15 secondi, ridisegno se cambia

Formato dei dati

- JSON dalle API, stessa origine
- Cookie di sessione + X-CSRF-TOKEN
- Dati dinamici sempre con escaping

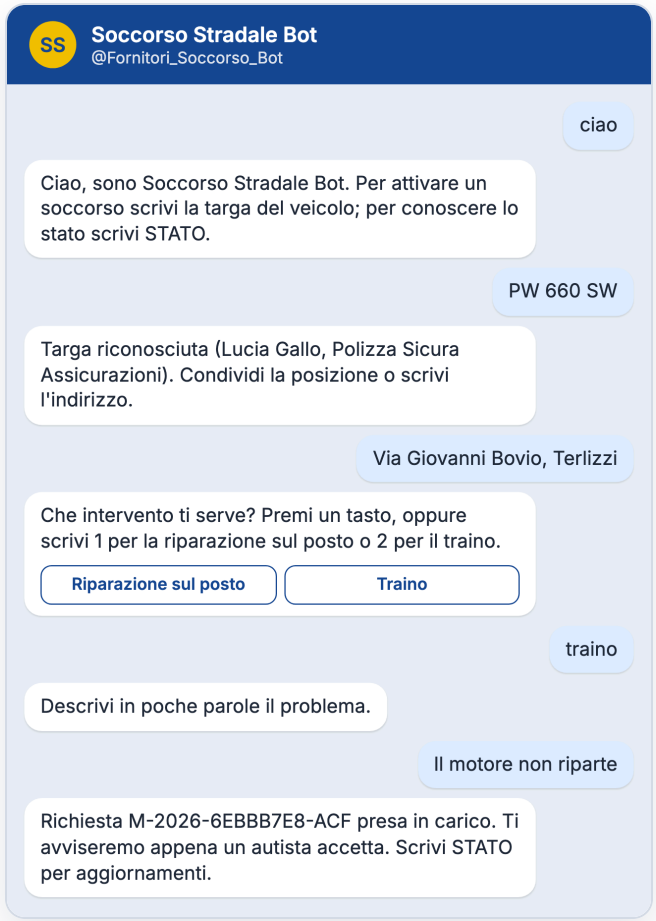
Membro 3

Chatbot

Marco Colucci

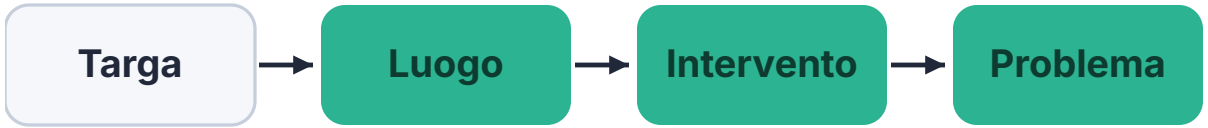
Soccorso Stradale Bot (@Fornitori_Soccorso_Bot) · dialogo guidato · riconoscimento targa · creazione della missione

Il dialogo con il cliente



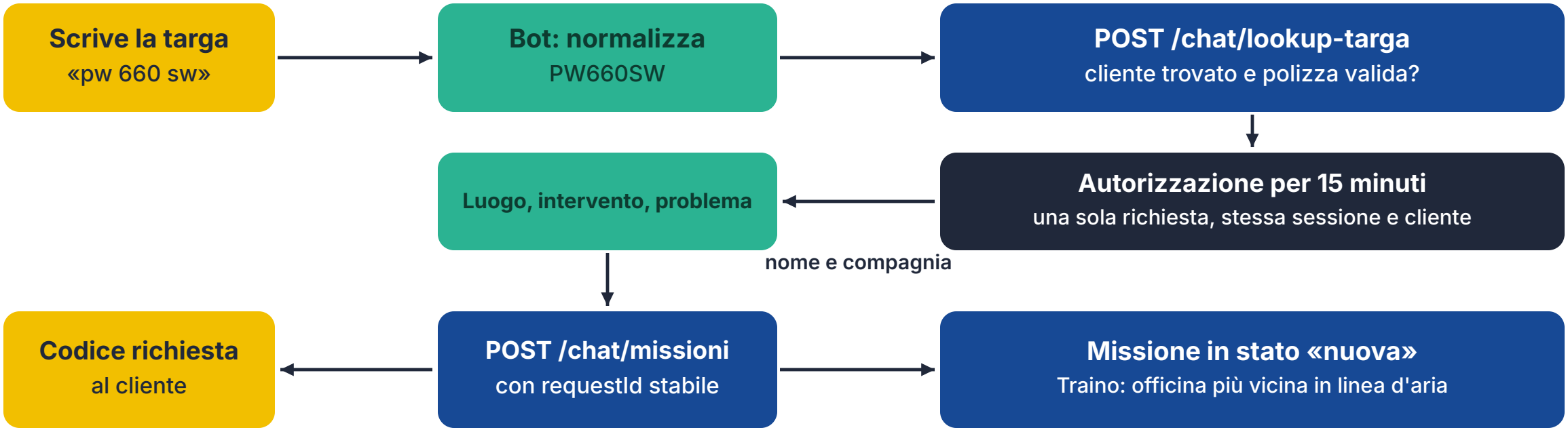
Testi reali, prodotti dal bot. Telegram simulato nella grafica.

- Una domanda alla volta
- Posizione condivisa o indirizzo scritto
- Riparazione o traino, con i tasti
- **AIUTO**, **ANNULLA**, **STATO** in ogni momento
- Linguaggio semplice, mai stati interni



Dialogo inattivo per 30 minuti: scade. 3 risposte non valide: ricomincia.

Riconoscimento della targa e creazione



5 targhe non trovate: blocco (429)
10 ricerche al minuto per sessione

requestId uguale nei tentativi: nessuna doppia missione

La targa **non prova l'identità**: limite demo V-01

Perché l'abbiamo fatto così

Organizzazione dei file

- **engine.py**: macchina a stati
- **messages.py**: tutti i testi
- **telegram_channel.py**: solo il canale

Pattern

- Macchina a stati del dialogo
- Backend come interfaccia: HTTP o finto
- InboundMessage: il canale è sostituibile

Formato dei dati

- JSON verso il backend, con token
- Targa normalizzata per la ricerca
- requestId e impronta SHA-256

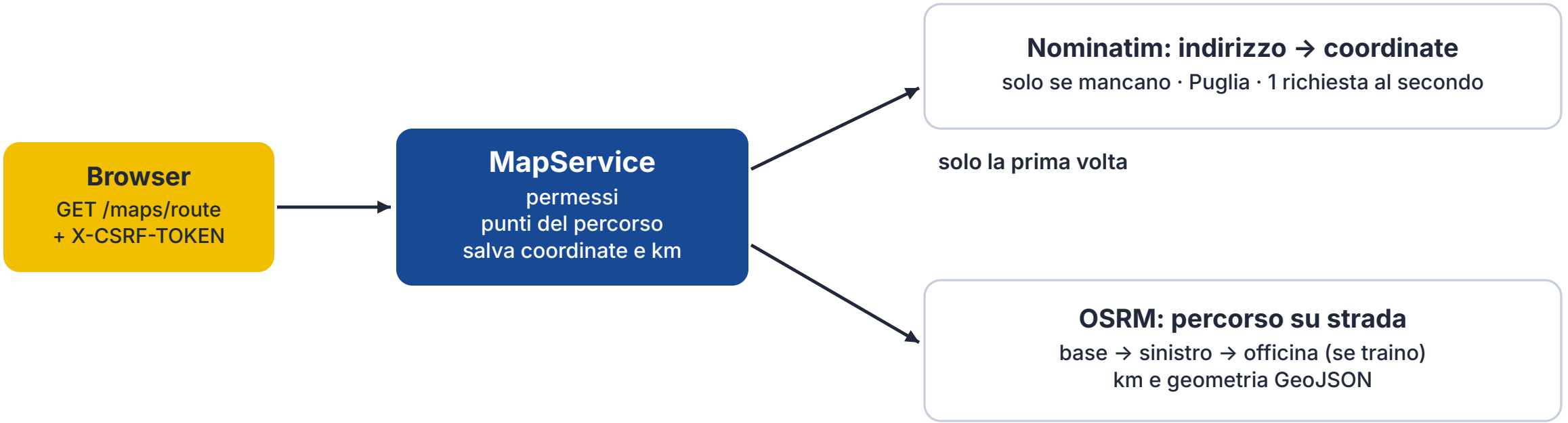
Membro 4

Mappe

Marco Colucci

Geocoding con Nominatim · percorso con OSRM · km stimati · mappa Leaflet

Geocoding e percorso



Esiti chiari

422 indirizzo non trovato
503 servizio non disponibile

Provider protetti

Timeout 3 s e 8 s
Niente redirect, max 1 MiB

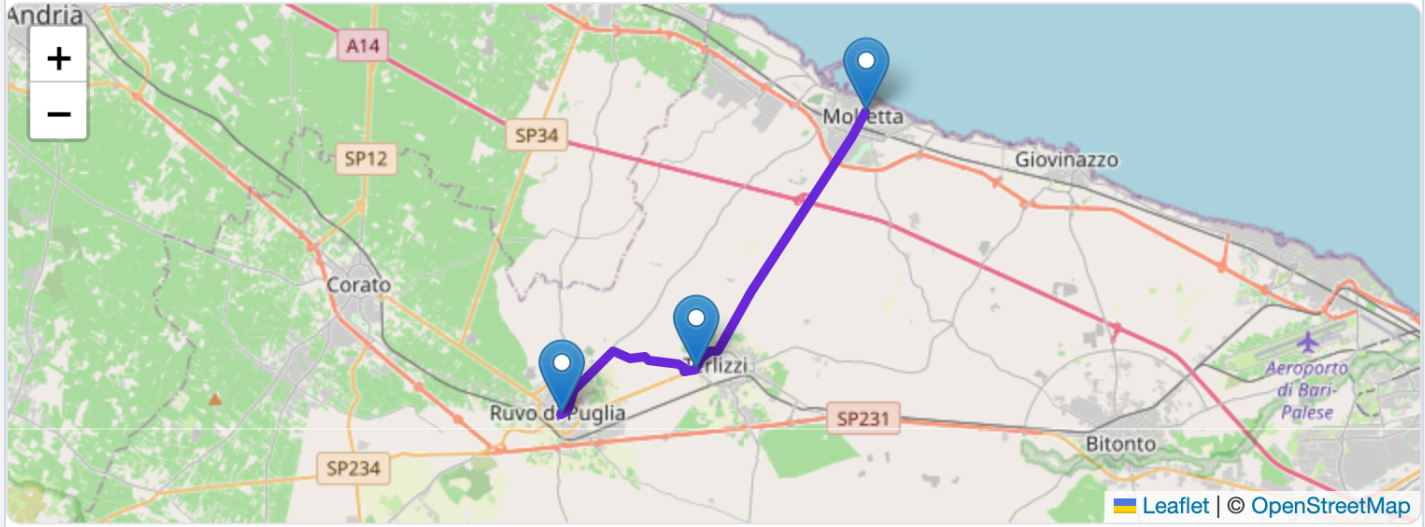
Il lavoro continua

I km si possono sempre inserire a mano

MAPPE

La mappa sullo schermo: Leaflet

Percorso stimato: 15.23 km (fonte: OSRM). I km effettivi si confermano alla chiusura.



1. Base operativa: Via dei Portuali 12 (indirizzo di esempio), Molfetta
2. Dove si trova il cliente: Via Giovanni Bovio, Terlizzi
3. Officina: Autofficina Murgia Service, Via Lama 3 (indirizzo di esempio), Ruvo di Puglia

Schermata reale: base a Molfetta, cliente a Terlizzi, officina a Ruvo

- Tile di **OpenStreetMap**, percorso **GeoJSON** di OSRM
- Segnaposto per base, cliente e officina
- Fonte e km stimati sempre indicati
- Autista: si apre da sola. Centrale: su richiesta

15,23 km
stimati da OSRM per questa missione

Perché l'abbiamo fatto così

Organizzazione dei file

- **MapController**: una sola API
- **MapService**: compone il percorso
- Un client per ciascun provider

Pattern

- Interfacce per i provider, sostituibili
- Backend come intermediario
- Nessuna transazione durante le chiamate

Formato dei dati

- GeoJSON per la geometria
- Coordinate NUMERIC(10,6)
- BigDecimal per i km stimati

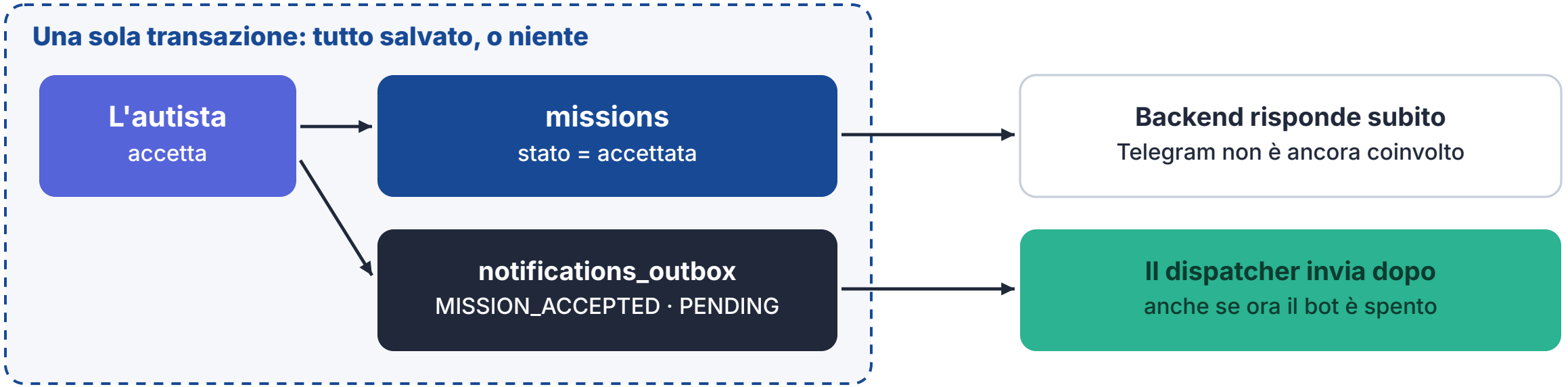
Membro 5

Notifiche

Marika Sancilio · Francesco Valeriano · Giuseppe Cangellosi

Outbox · dispatcher Telegram · stato di invio · consegna senza duplicati tra dispatcher

Outbox: la notifica nasce con la missione



event_key univoca

Lo stesso evento non si può registrare due volte

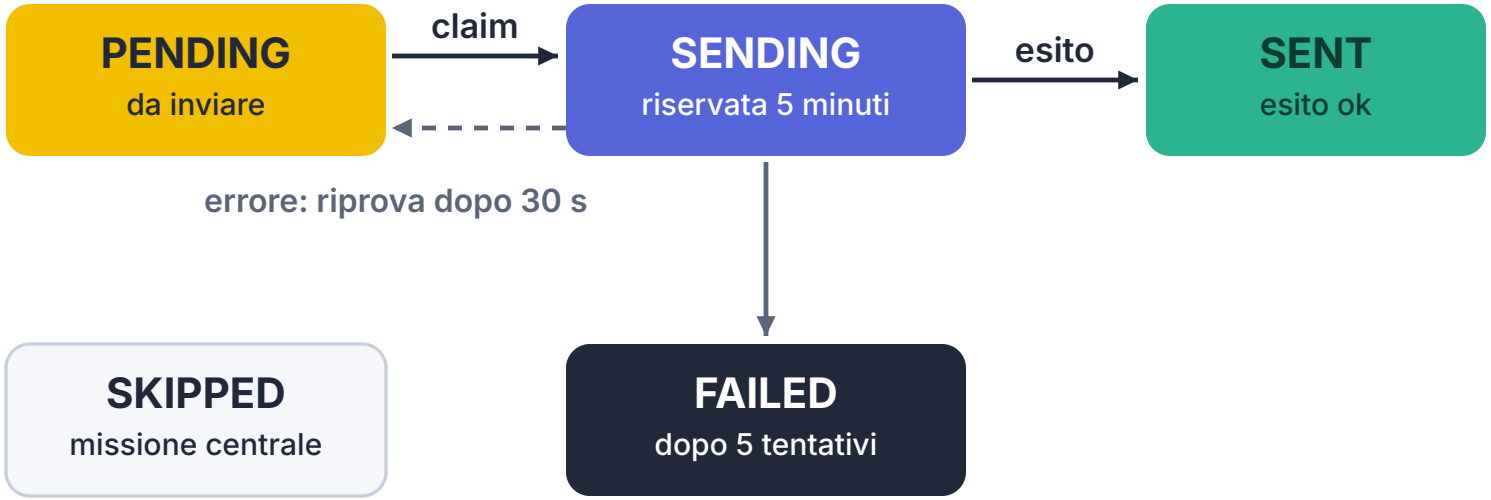
Payload minimo

Evento, codice, nome autista: nessun dato del cliente

Canale di origine

Telegram: PENDING. Centrale: SKIPPED

Dispatcher: consegna senza duplicati





Testi reali, grafica simulata

- **SKIP LOCKED**: due dispatcher non ricevono la stessa riga
- **Tentativo** nell'esito: un esito vecchio è ignorato
- **Riserva 5 minuti**: un bot caduto non blocca la coda
- **Limite**: crash tra invio ed esito, il messaggio può ripartire una volta

Perché l'abbiamo fatto così

Organizzazione dei file

- **NotificationOutbox** nel backend
- **notifications.py** nel bot
- API claim ed esito separate

Pattern

- Transactional outbox
- Riserva con scadenza (lease)
- Dispatcher indipendente dal canale

Formato dei dati

- Payload JSONB minimo
- Stati in maiuscolo, con CHECK
- tentativo per legare l'esito

CHIUSURA

Sicurezza e limiti noti

16	2	7
corrette	accettate per la demo	ancora aperte

Controlli attivi

- BCrypt, CSRF, ruoli, limiti ai tentativi
- CSP e corpo delle richieste limitato
- 139 test su database isolato

- **Sola targa** per la demo: V-01, aperta
- **WhatsApp** fuori ambito: canale Telegram
- HTTPS non provato su un server reale
- Dialogo in memoria: un riavvio lo azzera
- Provider mappe pubblici, senza garanzie

Demo con dati sintetici: non pronta per dati reali.

CHIUSURA

Scaletta della demo live

#	Dove	Cosa mostriamo
1	Sito pubblico	RecuVeicol e il collegamento al bot
2	Telegram	Targa, luogo, traino, problema: nasce la richiesta
3	Centrale	Scheda missione, **Proponi al più vicino**, conto alla rovescia
4	Autista (telefono)	Proposta e mappa, **Accetta**: compaiono i contatti
5	Telegram	Notifica automatica con il nome dell'autista, poi **STATO**
6	Autista	Chiusura con i km, costo calcolato dal server
7	Telegram e centrale	Notifica di conclusione, dashboard aggiornata

Se Telegram non risponde: bot da console sul backend reale. **Se le mappe non rispondono:** km inseriti a mano.

Sviluppi futuri

- **Identità del cliente**: secondo dato o codice SMS
- **WhatsApp e webhook**: già progettati nel contratto
- **Gestione utenti** dall'interfaccia
- **Messa in produzione**: HTTPS, ruoli DB separati, log
- **Qualità**: liste paginate, servizi più piccoli

Da confermare col docente

- Fascia notturna: 22:00 o 20:00
- Ora di riferimento della tariffa
- Canale: Telegram o anche WhatsApp
- Identità: basta la targa?

CHIUSURA

Conclusioni e domande

- Un sistema completo: dal messaggio alla chiusura
- Regole e costi solo sul server
- Componenti separati, contratto unico
- Notifiche affidabili con outbox
- Limiti noti, scritti e dichiarati

Domande?

Grazie per l'attenzione

RecuVeicol · ITS Developer 4.0